ERPLite — Full Parallel Build Prompt

Run with: claude (Claude Code terminal) from /home/opengrace/erplite

Or paste into VS Code Copilot agent / Antigravity IDE

Goal: what you build, you see — immediately, always

MISSION

You are building ERPLite — a production-grade ERP system at: `/home/opengrace/erplite`

The single most important rule of this build:

> **Frontend and backend are built simultaneously.** Every backend endpoint that is written must have a corresponding UI element that calls it and displays the result. No backend code exists without a visible UI counterpart. No UI component exists with a mock/hardcoded response.

The developer cannot read code. They navigate by what they see on `http://localhost:5173`. If it is not visible on screen, it does not exist for them.

WHAT MUST BE VISIBLE ON LOCALHOST:5173 WHEN DONE

Open the browser. You must see:

1. Persistent left sidebar — all 7 modules always visible

```
ERPLite                          [live dot] admin

— OPERATIONS —
  [green] StockFlow        [Live]
  [green] LedgerStock      [Live]

— MANUFACTURING —
  [amber] MaterialHub      [Phase 2]
  [amber] BuildOrder       [Phase 2]
  [amber] QualityGate      [Phase 2]

— COMMERCE —
  [gray]  DealFlow         [Phase 3]
  [gray]  RouteRunner      [Phase 3]
```

```
— SYSTEM —
  [green] Audit log        [Live]
  [green] Users & roles    [Live]
```

Live modules are clickable and fully functional.
Phase 2/3 modules are visible but show a clear "Coming in Phase 2/3" screen with the module description — they are NOT hidden or greyed out to the point of being invisible.

2. Top bar (always present)

- ERPLite logo (left)
- Current module name + tenant name
- WebSocket live indicator (pulsing green dot + "Live" text)
- Logged-in user avatar + name (right)

3. StockFlow module (WM) — fully functional

Three-panel layout:

LEFT PANEL (50% width) — Operation panel:

- Mode tabs: [Receiving] [Production] [Putaway] [Dispatch]
- Large barcode/HU scanner input (auto-focused, Enter triggers scan)
- Action buttons that change per mode:
  - Receiving: [Scan] [Create HU — coming soon]
  - Production: [Scan] [Consume] [Split]
  - Putaway: [Scan] [Move]
  - Dispatch: [Scan] [Pick — coming soon]
- Buttons disabled with tooltip when no HU scanned or wrong mode
- Quantity input appears inline when Consume is clicked
- Location dropdown appears inline when Move is clicked

CENTRE PANEL (25% width) — HU context card:
After a successful scan, show:

- HU code (large, monospace)
- Product name
- Quantity + unit (KG / IMP / QTY)
- Location (bin code)
- Status badge: green=AVAILABLE, amber=IN_USE, red=BLOCKED
- Lineage tree: parent HU → this HU → child HUs

RIGHT PANEL (25% width) — Live agent trace:

- "Agent activity" header with pulsing green dot
- Scrollable list of agent steps, newest at bottom, auto-scroll on
- Each step row shows: [AgentName] [ACTION] [STATUS badge] [timestamp]
  - SUCCESS = green background
  - BLOCKED = amber background
  - FAILED = red background
- Collapse/expand toggle
- "Only errors" filter toggle
- Clear button

BELOW panels — Inventory grid:

- Card per location: RECV-01, STOR-01, STOR-02, PROD-01
- Each card shows: location code, HU count, total quantity, status

4. LedgerStock module (IM) — fully functional

- Stock ledger view: table of all inventory_events (paginated)
- Columns: timestamp, event_type, HU code, qty change, location, actor
- Filter by: event type, date range, HU code
- Summary bar: total stock in, total consumed, net balance
- Real-time: new events appear at top without page refresh

5. Audit log — fully functional

- Table: timestamp, user, action, entity, before→after
- Paginated, 20 rows per page
- Filter by user and action type

6. Users & roles — functional

- List of users with role badges
- Admin can change roles inline

7. Coming soon screens (MaterialHub, BuildOrder, QualityGate, DealFlow, RouteRunner)

Each shows:

- Module name + ERPLite description

- SAP equivalent name (e.g. "Replaces SAP MM")
- Key capabilities list (3-5 bullet points)
- Phase badge + estimated availability
- "Notify me when live" button (non-functional placeholder)

TECHNOLOGY STACK (FIXED — DO NOT CHANGE)

Backend:

- Go 1.21+ with Gin framework
- PostgreSQL 16
- Redis 7 (idempotency + sessions)
- WebSocket via gorilla/websocket
- sqlc for type-safe DB queries
- Air for hot reload in dev

Frontend:

- React 19 + TypeScript (strict mode)
- Vite 5
- Zustand (global state: currentModule, currentHU, traceSteps, wsStatus, user)
- TanStack Query v5 (server state + caching)
- CSS custom properties only — no Tailwind, no styled-components
- No UI library — hand-written components only

Infrastructure:

- Docker Compose (postgres + redis + backend + frontend)
- Air (backend hot reload)
- Vite dev server (frontend hot reload)

PARALLEL BUILD RULE (ENFORCED)

Every feature is delivered in this order — NEVER backend-only first:

1. Write the DB migration / schema change
2. Write the sqlc query
3. Write the Go repository function

4. Write the Go service + agent orchestration
5. Write the API handler + route
6. Write the React component that calls this endpoint
7. Verify: open browser, perform the action, confirm it works visually

Steps 1-5 without step 6-7 = incomplete. Do not move to the next feature until the current one is visible and working in the browser.

WEBSOCKET — REAL-TIME FIRST

The WebSocket at `ws://localhost:8080/ws` is the heartbeat of the UI. It must be connected before any module is usable.

Frontend WebSocket hook (`/frontend/src/hooks/useWebSocket.ts`):

- Connect on app mount
- Auto-reconnect with exponential backoff (1s, 2s, 4s, max 8s)
- Show connection status in top bar:
  - Pulsing green dot = connected
  - Amber dot = reconnecting
  - Red dot = disconnected (show banner: "Connection lost — retrying...")
- On message type `agent_trace`: append to trace panel
- On message type `inventory_update`: refresh inventory grid + HU card
- On message type `event_posted`: refresh LedgerStock table

Backend WebSocket hub (`/backend/internal/api/handlers/ws_hub.go`):

- Single hub managing all connected clients
- Broadcast(msg) sends to all clients
- Each agent step calls hub.Broadcast() immediately as it executes
- Never batch — each step emits instantly

AGENT PIPELINE — EVERY OPERATION

All 8 agents must execute and emit for every operation.
No silent agents. No skipped steps.

Scan operation agent order:

1. BarcodeAgent → DECODE → RESOLVE

2. ErrorPreventionAgent → VALIDATE (duplicate scan check via Redis)
3. InventoryAgent → FETCH_HU
4. PricingAgent → RESOLVE_PRICE
5. AuditAgent → LOG

Consume operation agent order:

1. ErrorPreventionAgent → VALIDATE_STATE
2. ProductionAgent → VALIDATE_CONSUMPTION → SPLIT_HU
3. InventoryAgent → UPDATE_QUANTITY (parent) → CREATE_CHILD_HU
4. LabelingAgent → GENERATE_LABEL (version increment, NOT new identity)
5. AuditAgent → LOG

Move operation agent order:

1. ErrorPreventionAgent → VALIDATE_STATE
2. WarehouseAgent → CREATE_TASK → VALIDATE_LOCATION
3. InventoryAgent → UPDATE_LOCATION
4. AuditAgent → LOG

Each agent step MUST:

- Append to ExecutionTrace in context
- Call hub.Broadcast() with the step immediately
- Return AgentResult{Status, Data, Message}

DATABASE SCHEMA (COMPLETE — IMPLEMENT AS WRITTEN)

File: `/backend/db/migrations/001_init.sql`

Tables required (build all, even for Phase 2/3 modules — the schema is complete from day one; only the API and UI are phased):

```sql
```

```sql
-- Core identity
tenants (id BIGINT GENERATED ALWAYS AS IDENTITY PK,
         public_id UUID DEFAULT gen_random_uuid(),
         name TEXT NOT NULL, slug TEXT UNIQUE NOT NULL,
         created_at TIMESTAMPTZ DEFAULT now())

users (id BIGINT GENERATED ALWAYS AS IDENTITY PK,
       public_id UUID DEFAULT gen_random_uuid(),
       tenant_id BIGINT REFERENCES tenants(id),
       username TEXT NOT NULL, email TEXT,
       password_hash TEXT NOT NULL,
       created_at TIMESTAMPTZ DEFAULT now())

roles (id BIGINT GENERATED ALWAYS AS IDENTITY PK,
       name TEXT NOT NULL) -- ADMIN, WAREHOUSE_MANAGER, OPERATOR, VIEWER

user_roles (user_id BIGINT REFERENCES users(id),
            role_id BIGINT REFERENCES roles(id),
            PRIMARY KEY (user_id, role_id))

-- Material master
products (id BIGINT GENERATED ALWAYS AS IDENTITY PK,
          public_id UUID DEFAULT gen_random_uuid(),
          tenant_id BIGINT REFERENCES tenants(id),
          code TEXT NOT NULL, name TEXT NOT NULL,
          base_unit TEXT NOT NULL, -- KG / IMP / QTY
          description TEXT,
          attributes JSONB DEFAULT '{}',
          created_at TIMESTAMPTZ DEFAULT now())

batches (id BIGINT GENERATED ALWAYS AS IDENTITY PK,
         tenant_id BIGINT REFERENCES tenants(id),
         product_id BIGINT REFERENCES products(id),
         batch_number TEXT, expiry_date DATE,
         created_at TIMESTAMPTZ DEFAULT now())

-- Warehouse structure
warehouses (id BIGINT GENERATED ALWAYS AS IDENTITY PK,
            tenant_id BIGINT REFERENCES tenants(id),
            name TEXT NOT NULL, code TEXT NOT NULL)

locations (id BIGINT GENERATED ALWAYS AS IDENTITY PK,
           public_id UUID DEFAULT gen_random_uuid(),
           tenant_id BIGINT REFERENCES tenants(id),
           warehouse_id BIGINT REFERENCES warehouses(id),
           code TEXT NOT NULL, -- STOR-01, RECV-01, PROD-01
```

```
            zone TEXT, -- STORAGE / RECEIVING / PRODUCTION / DISPATCH
            capacity NUMERIC, unit TEXT,
            is_active BOOLEAN DEFAULT true)

-- Handling units (core entity)
handling_units (
  id BIGINT GENERATED ALWAYS AS IDENTITY PK,
  public_id UUID DEFAULT gen_random_uuid(),
  tenant_id BIGINT REFERENCES tenants(id),
  parent_hu_id BIGINT REFERENCES handling_units(id) ON DELETE SET NULL,
  product_id BIGINT REFERENCES products(id),
  batch_id BIGINT REFERENCES batches(id) ON DELETE SET NULL,
  location_id BIGINT REFERENCES locations(id),
  quantity NUMERIC NOT NULL CHECK (quantity >= 0),
  unit TEXT NOT NULL,
  status TEXT NOT NULL DEFAULT 'AVAILABLE',
    -- AVAILABLE / IN_USE / CONSUMED / CLOSED / BLOCKED
  label_version INT NOT NULL DEFAULT 1,
  created_at TIMESTAMPTZ DEFAULT now(),
  updated_at TIMESTAMPTZ DEFAULT now()
)

-- Barcodes (many barcodes can resolve to one entity)
barcodes (id BIGINT GENERATED ALWAYS AS IDENTITY PK,
          tenant_id BIGINT REFERENCES tenants(id),
          code TEXT NOT NULL,
          entity_type TEXT NOT NULL, -- HU / PRODUCT / LOCATION
          entity_id BIGINT NOT NULL,
          is_active BOOLEAN DEFAULT true,
          created_at TIMESTAMPTZ DEFAULT now(),
          UNIQUE (tenant_id, code))

-- Inventory event ledger (NEVER delete rows — append only)
inventory_events (
  id BIGINT GENERATED ALWAYS AS IDENTITY PK,
  tenant_id BIGINT REFERENCES tenants(id),
  event_type TEXT NOT NULL,
    -- GR / PUTAWAY / MOVE / CONSUME / SPLIT / MERGE / RELABEL / ADJUST
  hu_id BIGINT REFERENCES handling_units(id),
  child_hu_id BIGINT REFERENCES handling_units(id) ON DELETE SET NULL,
  from_location_id BIGINT REFERENCES locations(id) ON DELETE SET NULL,
  to_location_id BIGINT REFERENCES locations(id) ON DELETE SET NULL,
  quantity NUMERIC,
  unit TEXT,
  actor_user_id BIGINT REFERENCES users(id),
  reference_doc TEXT,
  metadata JSONB DEFAULT '{}',
```

```
  created_at TIMESTAMPTZ DEFAULT now()
) PARTITION BY RANGE (created_at);
-- Create initial partition:
CREATE TABLE inventory_events_2025
  PARTITION OF inventory_events
  FOR VALUES FROM ('2025-01-01') TO ('2026-01-01');
CREATE TABLE inventory_events_2026
  PARTITION OF inventory_events
  FOR VALUES FROM ('2026-01-01') TO ('2027-01-01');

-- Warehouse tasks
warehouse_tasks (
  id BIGINT GENERATED ALWAYS AS IDENTITY PK,
  public_id UUID DEFAULT gen_random_uuid(),
  tenant_id BIGINT REFERENCES tenants(id),
  task_type TEXT NOT NULL, -- PUTAWAY / MOVE / PICK / REPLENISH
  status TEXT NOT NULL DEFAULT 'OPEN', -- OPEN / IN_PROGRESS / CONFIRMED / CANCELLED
  hu_id BIGINT REFERENCES handling_units(id),
  from_location_id BIGINT REFERENCES locations(id) ON DELETE SET NULL,
  to_location_id BIGINT REFERENCES locations(id),
  assigned_to BIGINT REFERENCES users(id) ON DELETE SET NULL,
  opened_at TIMESTAMPTZ DEFAULT now(),
  confirmed_at TIMESTAMPTZ
)

-- Pricing (versioned, effective-date based)
price_versions (
  id BIGINT GENERATED ALWAYS AS IDENTITY PK,
  tenant_id BIGINT REFERENCES tenants(id),
  product_id BIGINT REFERENCES products(id),
  price NUMERIC NOT NULL CHECK (price >= 0),
  currency TEXT NOT NULL DEFAULT 'GBP',
  valid_from TIMESTAMPTZ NOT NULL,
  valid_to TIMESTAMPTZ, -- NULL means current
  created_by BIGINT REFERENCES users(id)
)

-- Agent trace system
execution_traces (
  id BIGINT GENERATED ALWAYS AS IDENTITY PK,
  trace_id UUID NOT NULL DEFAULT gen_random_uuid(),
  tenant_id BIGINT REFERENCES tenants(id),
  operation TEXT NOT NULL, -- SCAN / CONSUME / MOVE / SPLIT
  created_at TIMESTAMPTZ DEFAULT now()
)

execution_trace_steps (
```

```
  id BIGINT GENERATED ALWAYS AS IDENTITY PK,
  trace_id UUID NOT NULL,
  agent_name TEXT NOT NULL,
  action TEXT NOT NULL,
  status TEXT NOT NULL, -- SUCCESS / FAILED / BLOCKED
  payload JSONB DEFAULT '{}',
  step_order INT NOT NULL,
  created_at TIMESTAMPTZ DEFAULT now()
)

-- Audit log (immutable)
audit_log (
  id BIGINT GENERATED ALWAYS AS IDENTITY PK,
  tenant_id BIGINT REFERENCES tenants(id),
  actor_id BIGINT REFERENCES users(id),
  action TEXT NOT NULL,
  entity_type TEXT NOT NULL,
  entity_public_id UUID,
  payload_before JSONB,
  payload_after JSONB,
  created_at TIMESTAMPTZ DEFAULT now()
)

-- Blocked operations log
blocked_operations (
  id BIGINT GENERATED ALWAYS AS IDENTITY PK,
  tenant_id BIGINT REFERENCES tenants(id),
  reason TEXT NOT NULL,
  context JSONB DEFAULT '{}',
  created_at TIMESTAMPTZ DEFAULT now()
)

-- Phase 2 tables (schema ready, APIs not implemented yet)
-- MaterialHub (MM)
purchase_orders (id BIGINT GENERATED ALWAYS AS IDENTITY PK,
                 tenant_id BIGINT REFERENCES tenants(id),
                 supplier TEXT, status TEXT, created_at TIMESTAMPTZ DEFAULT now())
purchase_order_lines (id BIGINT GENERATED ALWAYS AS IDENTITY PK,
                      po_id BIGINT REFERENCES purchase_orders(id),
                      product_id BIGINT REFERENCES products(id),
                      qty_ordered NUMERIC, qty_received NUMERIC DEFAULT 0,
                      unit TEXT)

-- BuildOrder (PP)
production_orders (id BIGINT GENERATED ALWAYS AS IDENTITY PK,
                   tenant_id BIGINT REFERENCES tenants(id),
                   product_id BIGINT REFERENCES products(id),
```

```sql
                    planned_qty NUMERIC, actual_qty NUMERIC DEFAULT 0,
                    status TEXT DEFAULT 'PLANNED',
                    planned_start TIMESTAMPTZ, actual_start TIMESTAMPTZ,
                    created_at TIMESTAMPTZ DEFAULT now())
bom_lines (id BIGINT GENERATED ALWAYS AS IDENTITY PK,
           production_order_id BIGINT REFERENCES production_orders(id),
           product_id BIGINT REFERENCES products(id),
           required_qty NUMERIC, consumed_qty NUMERIC DEFAULT 0, unit TEXT)

-- QualityGate (QM)
inspection_lots (id BIGINT GENERATED ALWAYS AS IDENTITY PK,
                 tenant_id BIGINT REFERENCES tenants(id),
                 trigger_type TEXT, -- GR / PRODUCTION / SPLIT
                 hu_id BIGINT REFERENCES handling_units(id),
                 status TEXT DEFAULT 'PENDING', -- PENDING / PASS / FAIL
                 results JSONB DEFAULT '{}',
                 created_at TIMESTAMPTZ DEFAULT now())

-- DealFlow (SD)
sales_orders (id BIGINT GENERATED ALWAYS AS IDENTITY PK,
              tenant_id BIGINT REFERENCES tenants(id),
              customer TEXT, status TEXT DEFAULT 'DRAFT',
              total_value NUMERIC, currency TEXT DEFAULT 'GBP',
              created_at TIMESTAMPTZ DEFAULT now())
sales_order_lines (id BIGINT GENERATED ALWAYS AS IDENTITY PK,
                   so_id BIGINT REFERENCES sales_orders(id),
                   product_id BIGINT REFERENCES products(id),
                   qty NUMERIC, unit TEXT, unit_price NUMERIC)

-- RouteRunner (TM)
shipments (id BIGINT GENERATED ALWAYS AS IDENTITY PK,
           tenant_id BIGINT REFERENCES tenants(id),
           carrier TEXT, tracking_ref TEXT,
           status TEXT DEFAULT 'PLANNED',
           planned_dispatch TIMESTAMPTZ, actual_dispatch TIMESTAMPTZ,
           created_at TIMESTAMPTZ DEFAULT now())
```

Seed data (INSERT after all tables):

```sql
```

```sql
INSERT INTO tenants (name, slug) VALUES ('TechLogix UK', 'techlogix');
INSERT INTO roles (name) VALUES ('ADMIN'),('WAREHOUSE_MANAGER'),('OPERATOR'),('VIEWE
INSERT INTO users (tenant_id, username, email, password_hash)
  VALUES (1, 'admin', 'admin@techlogix.com', '$2a$10$placeholder_bcrypt_hash');
INSERT INTO user_roles VALUES (1,1);
INSERT INTO warehouses (tenant_id, name, code) VALUES (1,'Main Warehouse','WH01');
INSERT INTO locations (tenant_id, warehouse_id, code, zone)
  VALUES (1,1,'RECV-01','RECEIVING'),(1,1,'STOR-01','STORAGE'),
         (1,1,'STOR-02','STORAGE'),(1,1,'PROD-01','PRODUCTION');
INSERT INTO products (tenant_id, code, name, base_unit)
  VALUES (1,'FAB-001','Fabric Roll','KG'),(1,'LAM-001','Laminate','IMP');
INSERT INTO handling_units (tenant_id, product_id, location_id, quantity, unit)
  VALUES (1,1,2,315,'KG'),(1,2,3,21.16,'IMP');
INSERT INTO barcodes (tenant_id, code, entity_type, entity_id)
  VALUES (1,'HU-1001','HU',1),(1,'HU-1002','HU',2);
INSERT INTO price_versions (tenant_id, product_id, price, currency, valid_from)
  VALUES (1,1,4.50,'GBP',now()),(1,2,2.75,'GBP',now());
```

API ENDPOINTS (IMPLEMENT ALL)

Auth

```
POST /api/auth/login    → { username, password } → JWT in HTTP-only cookie
POST /api/auth/logout   → clear cookie
GET  /api/auth/me       → { user, roles, tenant }
```

StockFlow (WM)

```
POST /api/scan          → { barcode } → { hu, trace }
POST /api/consume       → { hu_public_id, quantity } → { parent_hu, child_hu,
trace }
POST /api/move          → { hu_public_id, to_location_code } → { hu, task, trace }
POST /api/split         → { hu_public_id, quantity } → { parent_hu, child_hu,
trace }
GET  /api/hus           → paginated list of all HUs
GET  /api/hus/:id       → single HU with full detail
GET  /api/hus/:id/lineage → { parent, self, children[] }
GET  /api/locations     → all locations with HU counts + total quantities
```

LedgerStock (IM)

```
GET /api/inventory/events    → paginated inventory_events (filter: type, date, hu)
```

```
GET /api/inventory/snapshot  → current stock per product per location
GET /api/inventory/summary   → { total_in, total_consumed, net_balance } per
product
```

System

```
GET  /api/trace/:trace_id  → full trace with all steps
GET  /api/audit            → paginated audit_log (admin only)
GET  /api/users            → user list with roles (admin only)
PUT  /api/users/:id/role   → change role (admin only)
GET  /health               → { status: "ok", db: "ok", redis: "ok", ws_clients: N
}
GET  /ws                   → WebSocket upgrade
```

FRONTEND FILE STRUCTURE (EXACT)

```
/frontend/src/
├── api/
|   └── client.ts           ← axios instance, all typed API methods
├── hooks/
|   ├── useWebSocket.ts     ← WS connection, auto-reconnect, message routing
|   ├── useScan.ts          ← scan logic: POST /scan → update store
|   └── useInventory.ts     ← TanStack Query wrappers for inventory endpoints
├── store/
|   └── useAppStore.ts      ← Zustand: currentModule, currentHU, traceSteps,
|                               wsStatus, user, currentMode
├── components/
|   ├── layout/
|   |   ├── Sidebar.tsx     ← all 7 modules, nav state, phase badges
|   |   ├── TopBar.tsx      ← logo, module name, WS status, user
|   |   └── Shell.tsx       ← grid layout: sidebar + main + trace
|   ├── stockflow/
|   |   ├── OperationPanel.tsx   ← mode tabs, scanner, action buttons
|   |   ├── ModeSelector.tsx
|   |   ├── ScanInput.tsx        ← auto-focus, Enter key
|   |   ├── ActionButtons.tsx    ← mode-aware enable/disable + tooltips
|   |   ├── ContextPanel.tsx     ← HU card after scan
|   |   ├── HUCard.tsx
|   |   ├── LineageTree.tsx
|   |   └── InventoryGrid.tsx    ← location cards
|   ├── ledgerstock/
|   |   ├── EventLedger.tsx      ← paginated events table
|   |   ├── StockSnapshot.tsx    ← current stock per product/location
|   |   └── StockSummary.tsx     ← summary bar
```

```
|   ├── trace/
|   |   ├── AgentTracePanel.tsx  ← scrollable step list
|   |   └── TraceRow.tsx         ← single step with colour coding
|   ├── comingsoon/
|   |   └── ComingSoonModule.tsx ← reusable coming-soon screen
|   └── system/
|       ├── AuditLog.tsx
|       └── UserManagement.tsx
└── pages/
    ├── LoginPage.tsx
    └── AppPage.tsx             ← shell + module routing
```

SECURITY (NON-NEGOTIABLE)

- JWT: RS256, 15-minute access token, 7-day refresh in HTTP-only cookie
- RBAC: middleware checks role before every protected route
- Input validation: every POST body validated with Go struct tags
- SQL: sqlc-generated queries only — no string concatenation
- Redis idempotency:
  - Scan: key `SCAN:{tenant}:{barcode}` TTL 5s
  - Mutations: key `OP:{tenant}:{hu_id}:{op}` TTL 30s
- Audit: before+after state logged for every mutation
- No raw internal IDs in API responses — always public_id (UUID)

DOCKER COMPOSE (COMPLETE)

File: `/home/opengrace/erplite/docker-compose.yml`

```yaml
```

```yaml
services:
  postgres:
    image: postgres:16-alpine
    environment:
      POSTGRES_DB: erplite
      POSTGRES_USER: erplite
      POSTGRES_PASSWORD: erplite_dev
    volumes:
      - pg_data:/var/lib/postgresql/data
      - ./backend/db/migrations:/docker-entrypoint-initdb.d:ro
    ports: ["5432:5432"]
    healthcheck:
      test: ["CMD-SHELL","pg_isready -U erplite"]
      interval: 5s
      retries: 10

  redis:
    image: redis:7-alpine
    ports: ["6379:6379"]
    healthcheck:
      test: ["CMD","redis-cli","ping"]
      interval: 5s
      retries: 5

  backend:
    build: { context: ./backend, dockerfile: Dockerfile.dev }
    environment:
      DB_DSN: postgres://erplite:erplite_dev@postgres:5432/erplite?sslmode=disable
      REDIS_ADDR: redis:6379
      PORT: "8080"
      JWT_PRIVATE_KEY_PATH: /app/keys/private.pem
      JWT_PUBLIC_KEY_PATH: /app/keys/public.pem
      GIN_MODE: debug
    ports: ["8080:8080"]
    volumes:
      - ./backend:/app
    depends_on:
      postgres: { condition: service_healthy }
      redis:    { condition: service_healthy }

  frontend:
    build: { context: ./frontend, dockerfile: Dockerfile.dev }
    environment:
      VITE_API_URL: http://localhost:8080
      VITE_WS_URL: ws://localhost:8080/ws
    ports: ["5173:5173"]
```

```
    volumes:
      - ./frontend:/app
      - /app/node_modules

volumes:
  pg_data:
```

backend/Dockerfile.dev:

```dockerfile
FROM golang:1.21-alpine
RUN go install github.com/cosmtrek/air@latest
RUN apk add --no-cache git
WORKDIR /app
COPY go.mod go.sum ./
RUN go mod download
CMD ["air", "-c", ".air.toml"]
```

frontend/Dockerfile.dev:

```dockerfile
FROM node:20-alpine
WORKDIR /app
COPY package*.json ./
RUN npm install
CMD ["npm", "run", "dev", "--", "--host", "0.0.0.0", "--port", "5173"]
```

SUCCESS CRITERIA — SYSTEM IS DONE WHEN ALL PASS

Run: `docker compose up --build` Open: `http://localhost:5173`

Visual checks (in order):

1. Login page appears — enter admin / admin123 — redirected to app
2. Sidebar shows all 7 modules with correct phase badges
3. Top bar shows "ERPLite · TechLogix UK" + pulsing green dot
4. StockFlow opens by default — mode tabs visible, scanner input focused

Functional checks:

5. Type `HU-1001` + Enter → agent trace fires 5 steps → HU card appears
6. HU card shows: Fabric Roll, 315 KG, STOR-01, AVAILABLE

7. Switch to Production mode → Consume and Split buttons enable
8. Click Consume → quantity input appears → enter 50 → confirm
9. Agent trace shows: ProductionAgent:SPLIT_HU:EXECUTED → child HU created
10. HU card updates: HU-1001 now 265 KG
11. Inventory grid updates: STOR-01 shows updated quantity
12. Scan HU-1001 again immediately → ErrorPreventionAgent:DUPLICATE_SCAN:BLOCKED in amber
13. Switch to Putaway mode → Consume/Split disabled with "Not available in Putaway mode"
14. Click LedgerStock in sidebar → event table shows the SPLIT event just posted
15. Click MaterialHub → Coming Soon screen with "Phase 2 · 90 days" badge
16. Click Audit log → table shows all actions taken
17. Disconnect Docker (stop backend) → top bar shows amber dot + "Reconnecting..."
18. Restart backend → green dot returns, UI resumes without page refresh

Build checks:

- `go build ./...` → 0 errors
- `go test ./...` → all pass
- `npm run build` → 0 TypeScript errors

ABSOLUTE RULES

1. No V1/V2 phasing within the codebase. Build it right the first time.
2. No placeholder functions that return hardcoded data. Every response comes from the real database.
3. No microservices. One Go binary. One PostgreSQL. One Redis.
4. No silent agents. Every agent step emits a WebSocket broadcast.
5. HU identity is never broken. Split creates a child with parent_hu_id. The parent retains its ID and increments label_version.
6. Stock is never overwritten. Only inventory_events are appended.
7. Frontend never reads the database directly. Always via API.
8. Coming soon modules are visible, not hidden.
9. Every number displayed is rounded. No floating point artifacts.
10. The developer judges by what they see. If it is not visible, it does not exist.